Data Driven Control Integration Week

Group 8 (MPC)

Christian Brunet Jules Marceau
Matthieu Moussaoui Clément Tamboise
Rayane Trigoust

CentraleSupélec

Friday, November 6, 2026

Table of Contents

1. Introduction
2. Dataset Validation
3. System Modeling
4. Z-axis control
5. Advanced Control Law

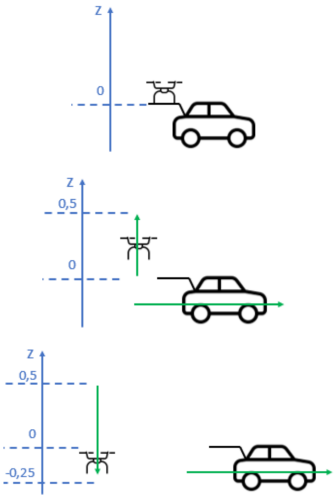
Project Objectives

Available Resources:

- Dataset of vertical acceleration.
- MATLAB/Simulink drone simulator.

Objectives:

- Validate the dataset.
- Identify the model.
- Develop a control law.



Dataset Validation

Cleaning the data

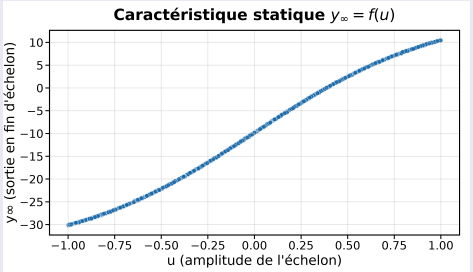
Remove lines that includes:

- Frozen sensor for 2+ seconds
- 5+ diverging values

Normalisation

- Directly in the training

Static characteristic



$y_\infty = f(u)$ after eliminating corrupted runs.

System Modeling

Linear Model

State Function

First-order lag driven by $N(u)$:

$$\begin{cases} \dot{x}(t) = -\dfrac{x(t)}{\tau} + \dfrac{N(u(t))}{\tau} \\ a_z(t) = x(t) \end{cases}$$

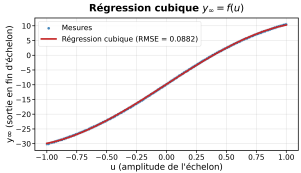
Parameter Identification

- **Static characteristic** $N(u)$: 3rd-order polynomial identified on steady-state regimes:

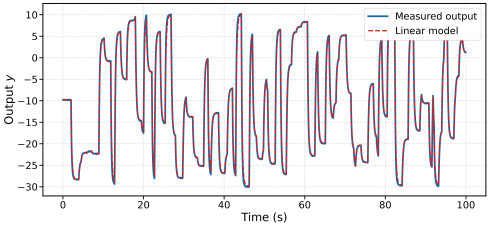
$$a_z = N(u) = a_3 u^3 + a_2 u^2 + a_1 u + a_0$$

- **Tau identification**

$a_3 \approx -6.05$, $a_2 \approx 1.1\times10^{-3}$, $a_1 \approx 26.1$, $a_0 \approx -9.81$, $\tau \approx 0.22$ s



Linear Model Validation



First RNN

Sequence-to-sequence architecture

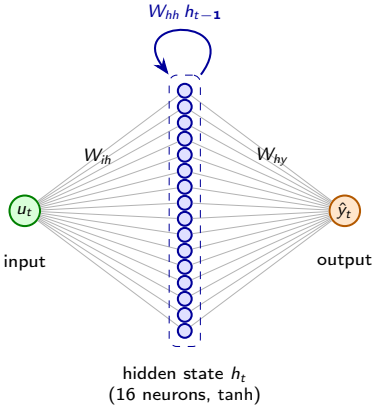
Parameters

- One scalar input u_t and one scalar output $\hat{y}_t$
- Hidden state dimension: 16
- Early stopping to avoid overfitting

State representation

$$h_t = \tanh(W_{ih}u_t + W_{hh}h_{t-1} + b)$$

$$\hat{y}_t = W_{hy}h_t + b_y$$



First RNN

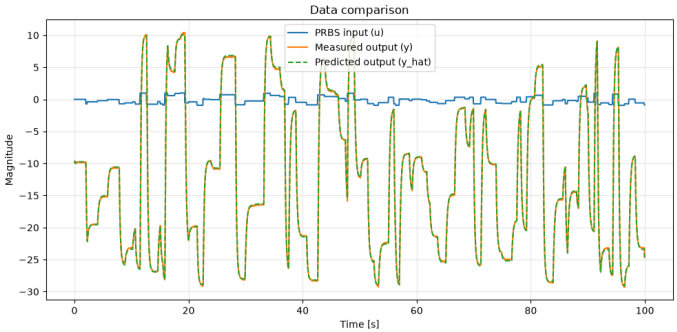
Results

Numerical results

- Validation loss: 3.93×10^{-4}
- Test loss: 4.01×10^{-4}

Training criterion

$$\mathcal{L} = \frac{1}{N} \sum_{i=1}^{N} (y_i - \hat{y}_i)^2$$



Z-axis control

Designing the reference trajectory

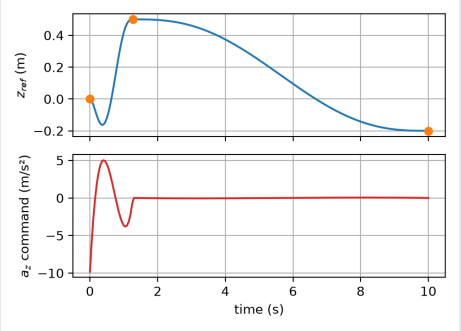
Principle

- **Input**: waypoints z_i and passage times t_i
- One **polynomial function** per segment $[t_i, t_{i+1}]$, imposing z, $\dot{z}$ and $\ddot{z}$ at both ends
- Null speed at waypoints, null acceleration on arrival, $\ddot{z}$ continuous
- **Output**: $z_{ref}(t)$ and the acceleration command $\ddot{z}_{ref}(t)$

Results

Smooth analytic reference; overshoot beyond waypoints removed by shortening segments

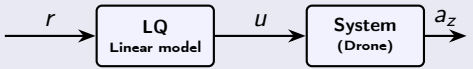
Example: $z = 0, 0.5, -0.2$ m at $t = 0, 5, 10$ s



LQ control

Linear-quadratic control design

Control principle



Performance

- Global trajectory following
- Wrong amplitude + phase shift
- Missing prediction

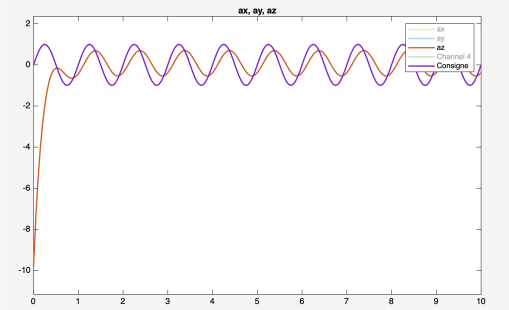
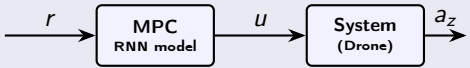


Figure: LQ control response for a sinusoidal reference.

MPC control

Model predictive control: results

Control principle



Performance

- Good overall tracking of the reference.
- No phase lag: good RNN prediction.
- Full amplitude not reached.
- Open loop: no measurement feedback.

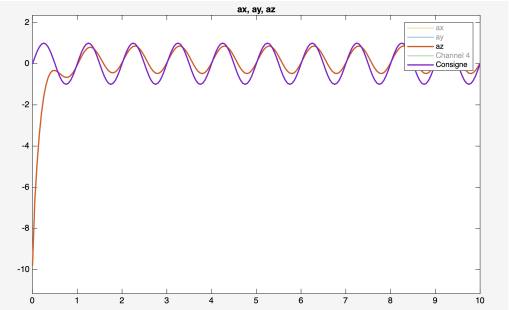


Figure: MPC-RNN response for a sinusoidal reference.

MPC control

Model predictive control design

Cost function minimized

$$J(U) = \sum_{i=1}^{N} q \left(\hat{a}_{z,k+i} - r_{k+i}\right)^2 + \rho\, u_{k+i-1}^2$$

$U = (u_k, \ldots, u_{k+N-1})$, $-1 \leq u_{k+i} \leq 1$, $N = 20$
$\hat{a}_z$: acceleration predicted by the RNN from its internal state
Only u_k is applied, then the horizon slides

J is minimized by gradient descent with Adam steps (gradients through the RNN by autodiff)

From a_z to z

- First tried: control based on a_z.
- a_z followed the reference, but z diverged from the request: small errors on a_z accumulate when integrated twice.
- So the cost was moved to the height:

$$J(U) = \sum_{i=1}^{N} q \left(\hat{z}_{k+i} - r_{k+i}\right)^2 + \rho \left(u_{k+i-1} - u_{k+i-2}\right)^2$$

Extended Kalman Filter

Estimator built on the RNN model

Model

State $x_k = (h_k,\ v_{z,k},\ z_k)$: RNN hidden state, vertical velocity, height.

$$x_k = f(x_{k-1}, u_{k-1}) + w_k, \qquad w_k \sim \mathcal{N}(0, Q)$$
$$y_k = Hx_k + c + v_k, \qquad v_k \sim \mathcal{N}(0, R)$$

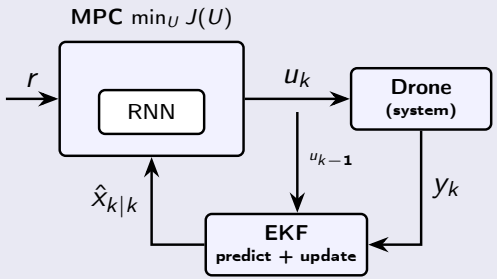
The function f

$$h_k = \tanh(W_x h_{k-1} + W_u u_{k-1} + b) \quad \text{RNN step}$$
$$v_{z,k} = v_{z,k-1} + a_{z,k}\,\Delta t \quad \text{integrate}$$
$$z_k = z_{k-1} + v_{z,k}\,\Delta t \quad \text{integrate}$$

$a_{z,k} = W_f h_k + b_f$ is encoded in h_k (linear readout of the RNN), so it is not a separate state.

Measured: $y_k = (a_{z,k},\ v_{z,k},\ z_k)$, so $c = (b_f, 0, 0)$.

Control principle



$y_k = (a_z, v_z, z)$: measurements

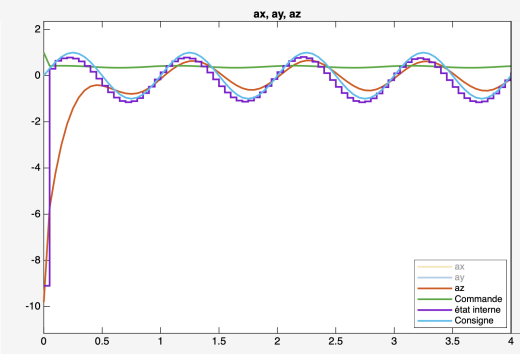
$\hat{x}_{k|k} = (\hat{h}, \hat{v}_z, \hat{z})$: estimated state, initial state of the RNN prediction at each step

MPC without vs with EKF

Tracking of a sinusoidal a_z reference

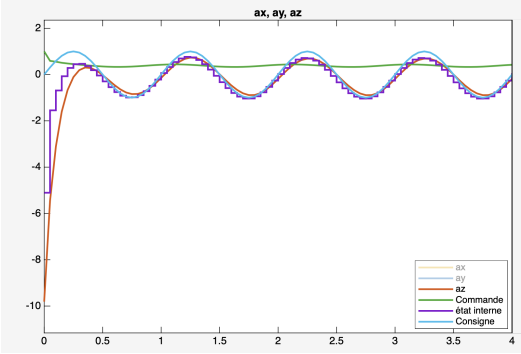
Without EKF

open loop: RNN internal state only



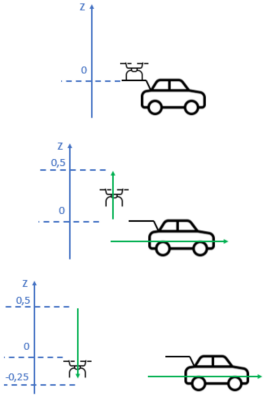
With EKF

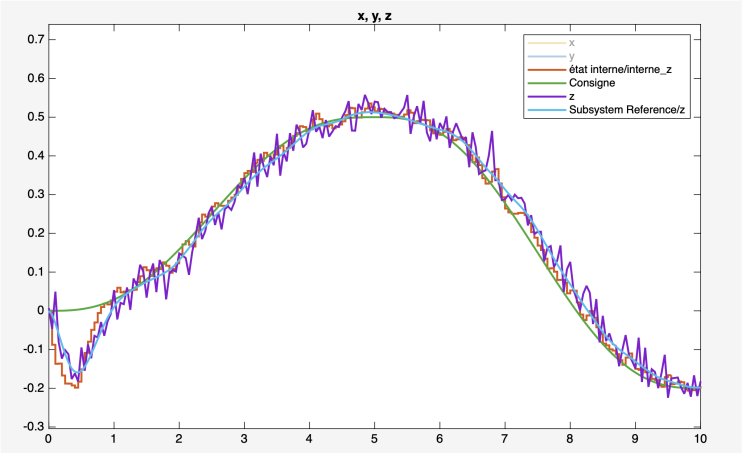
state corrected by the measurements



Requested command

MPC control with EKF feedback





Comparison

LQ, MPC and MPC with EKF

LQ control with Linear Model

- Simple and very fast.
- Lack prediction.
- Phase shifting.

MPC control with RNN

- Prediction.
- Internal state diverges from reality.
- No feedback loop.

MPC with EKF

- Feedback from measurements.
- Position control.

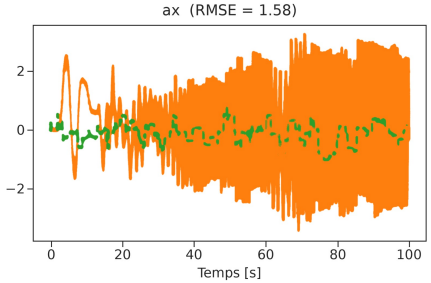
Data generation

First attempt: PRBS directly on the motor commands

Objective

Generate rich input/output trajectories to identify a nonlinear dynamic model of the drone in 3D (4 inputs and 12 outputs).

- Sampling time: $T_s = 0.05\,\mathrm{s}$
- $N = 2000$ samples per trajectory
- 3000 independent experiments
- Hover command: $U_{\mathrm{hover}} = 0.55$
- Four independent piecewise-constant modes $\delta = [\delta_T, \delta_r, \delta_p, \delta_y]$, held for 1 to 3 s around hover



Why it was not enough

Without feedback the drone is unstable: it does not stay near hover, so most trajectories leave the useful flight domain. The data is unusable for identification.

$\Rightarrow$ put a flight controller in the loop.

Data generation

With a flight controller in the loop

APRBS on the references

We no longer excite $U_1, \ldots, U_4$ directly. The four references of the flight controller are excited instead:

$$r = [z_{\text{ref}},\ \phi_{\text{ref}},\ \theta_{\text{ref}},\ \psi_{\text{ref}}]$$

Each level is drawn uniformly and held for a random duration between 1 and 3 seconds.

Dataset

- 3000 simulations of 100 s, sampled at 20 Hz
- First 3 s dropped (motor start-up)
- Runs with an angle above 1 rad rejected (drone lost)
- New sensor-noise seeds for every run

From version 1 to version 2

Version 1 used PID controllers directly on the attitude angles: stable, but not fast enough (see the appendix). Version 2 is a cascade controller: an outer angle loop and a fast inner angular-rate loop.

Controller parameters

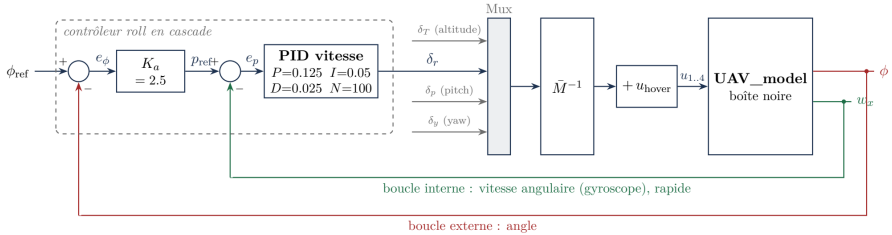
- Outer angle gain: $K_a = 2.5$
- Inner rate PID:
  $P = 0.125,\ I = 0.05,\ D = 0.025,\ N = 100$
- Sampling time: $T_s = 0.005\,\text{s}$
- Same structure for roll and pitch

Data generation

Flight controller — version 2

Contrôle d'attitude en cascade : angle puis vitesse angulaire

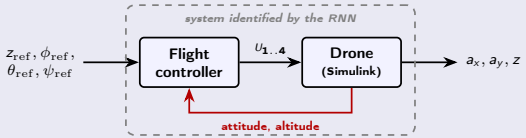
Axe roll. Le pitch est identique avec θ, w_y et δ_p.



New RNN model

Trained on flight controller + drone simulations

What the RNN learns



Model and training

- Elman RNN, 32 hidden units, 1.3 k parameters
- 4 inputs $\rightarrow$ 3 outputs (a_x, a_y, z)
- 3000 trajectories of 1941 steps, 70 % for training
- Adam, $\text{lr} = 10^{-3}$, 100 epochs, batch 32
- Test RMSE: a_x 0.24, a_y 0.23 m/s^2, z 0.048 m

Why a new model?

- The first RNN only described the z axis $(u \rightarrow a_z)$.
- 3D control needs x and y, but raw motor data is unusable (unstable drone).
- With the flight controller inside, the identified system is stable, with bounded physical inputs.
- The MPC sends references to the flight controller, like an autopilot.

Choice of the outputs

- z is held by the altitude loop: learned directly, no double integration (lesson from a_z).
- No position loop on x, y: learn a_x, a_y, then integrate.

3D MPC

Position control through the flight controller

Cost function minimized

$$J(U) = \sum_{i=1}^{N} Q \left\| \hat{p}_{k+i} - r_{k+i} \right\|^2 + R \left\| u_{k+i-1} - u_{k+i-2} \right\|^2$$

$\hat{p} = (\hat{x}, \hat{y}, \hat{z})$: position predicted with the RNN
$u = (z_{\text{ref}}, \phi_{\text{ref}}, \theta_{\text{ref}}, \psi_{\text{ref}})$: references sent to the flight controller
r: 3D reference trajectory

Prediction over the horizon

$$(a_x, a_y, z)_k = \text{RNN}(h_{k-1}, u_{k-1})$$
$$v_k = v_{k-1} + a_k \, \Delta t \quad (x, y)$$
$$p_k = p_{k-1} + v_k \, \Delta t \quad (x, y)$$

z is read directly from the RNN output.

Constraints

$u_{\min} \leq u \leq u_{\max}$: the range seen in training

- $z_{\text{ref}} \in [-1,\ 2.5]$ m
- $\phi_{\text{ref}}, \theta_{\text{ref}} \in [-0.3,\ 0.3]$ rad
- $\psi_{\text{ref}} \in [-0.5,\ 0.5]$ rad

Outside this range the RNN would extrapolate.

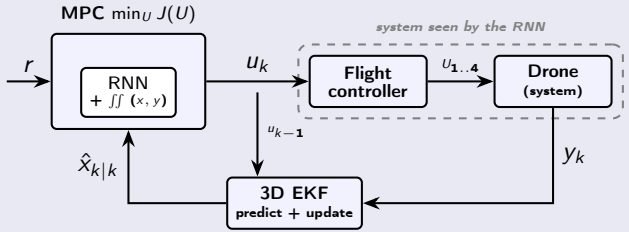
Settings

- 50 Adam steps per sample ($\text{lr} = 0.05$), warm start
- $T_e = 0.05$ s, $Q = R = 1$
- $N = 20$ (1 s): with $N = 10$ (0.5 s) the MPC diverges

3D control loop

MPC on top of the flight controller, state estimated by a 3D EKF

Control principle



$r = (x_r, y_r, z_r)$: 3D reference (car position, z_r offset)
$u = (z_{\text{ref}}, \phi_{\text{ref}}, \theta_{\text{ref}}, \psi_{\text{ref}})$: references sent to the flight controller
$y_k = (a_x, a_y, z, v_x, v_y, x, y)$: measurements

3D EKF

State $x_k = (h_k,\ v_{x,k},\ v_{y,k},\ x_k,\ y_k)$

$$h_k = \tanh(W_x h_{k-1} + W_u u_{k-1} + b)$$

$$v_k = v_{k-1} + a_k\,\Delta t$$

$$p_k = p_{k-1} + v_k\,\Delta t$$

Same model as the MPC prediction: (a_x, a_y, z) is the linear readout of h_k, so z needs no extra state.

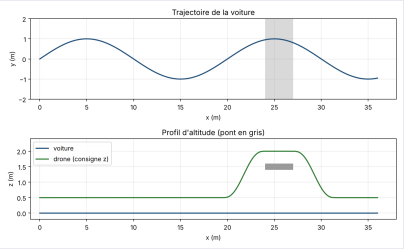
Role

- Resets the RNN hidden state and the integrated v, p at each step
- Removes the drift of the double integration on x, y

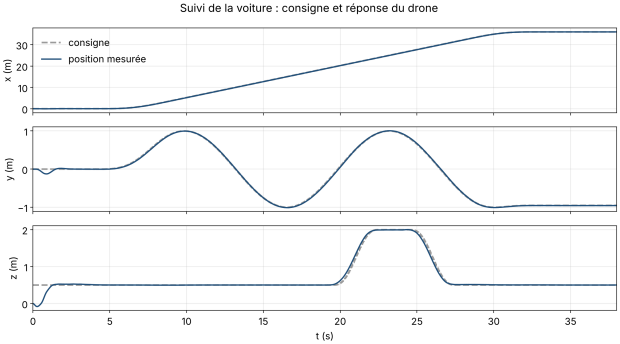
Results: following the car

3D MPC + EKF on the Simulink drone with its flight controller

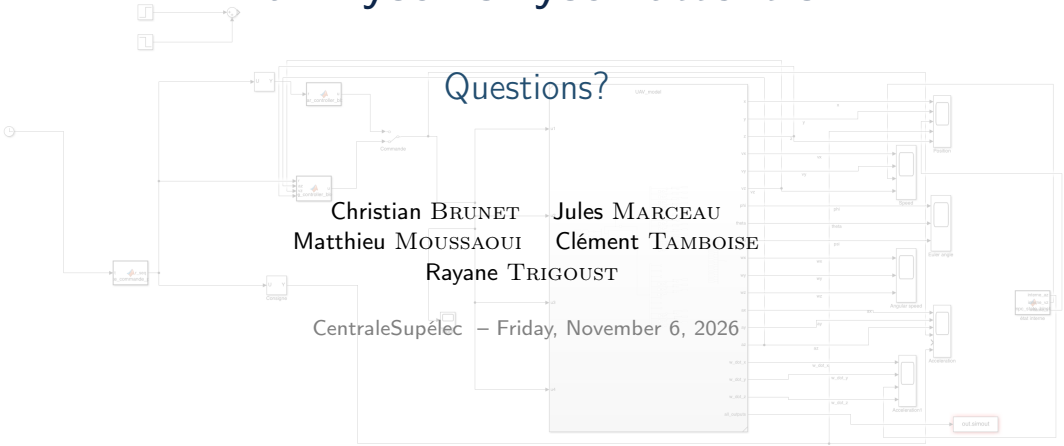
Scenario



Sinusoidal car path; the drone flies 0.5 m above it and climbs to 2 m to pass over the bridge ($x \in [24, 27]$ m).



Thank you for your attention



Questions?

Christian Brunet Jules Marceau
Matthieu Moussaoui Clément Tamboise
Rayane Trigoust

CentraleSupélec – Friday, November 6, 2026

Appendix: System Modeling

Linear Model: Least Square Method

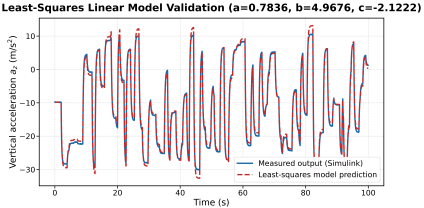
Model equations

$$y_k(t+1) = a\,y_k(t) + b\,u_k(t) + c$$
$$= \varphi_k^{\mathsf{T}}\theta.$$

$$\varphi_k^{\mathsf{T}} = \begin{bmatrix} y_k(t) & u_k(t) & 1 \end{bmatrix} \qquad \theta = \begin{bmatrix} a & b & c \end{bmatrix}^{\mathsf{T}}.$$

The least-squares estimate is

$$\theta^{\star} = \left(\Phi^{\mathsf{T}}\Phi\right)^{-1}\Phi^{\mathsf{T}}Y.$$



Appendix: System Modeling

Linear Model with Constant Steady-State Gain

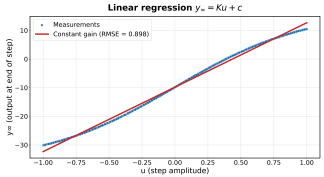
State Function

First-order lag driven by a constant gain K and offset c: $\dot{v}(t) = \frac{1}{\tau}\big(Ku(t) - v(t)\big)$, i.e. $\tau\,\dot{v} + v = Ku + c$.

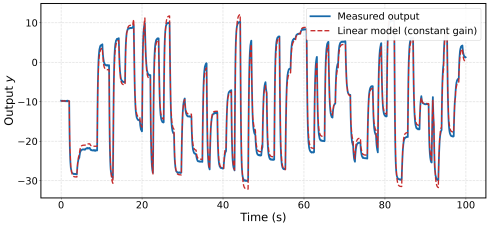
Parameter Identification

- **Static gain** K: affine least-squares fit on steady-state regimes:

$$v(t) = K\,u(t) + c, \qquad K \approx 22.5,\ c \approx -9.8$$



Linear Model Validation



Appendix: Extended Kalman Filter

Predict and update steps

Predict

$$\hat{x}_{k|k-1} = f(\hat{x}_{k-1|k-1}, u_{k-1})$$
$$P_{k|k-1} = F_k P_{k-1|k-1} F_k^\top + Q, \quad F_k = \frac{\partial f}{\partial x}$$

Update

$$K_k = P_{k|k-1} H^\top (H P_{k|k-1} H^\top + R)^{-1}$$
$$\hat{x}_{k|k} = \hat{x}_{k|k-1} + K_k (y_k - H\hat{x}_{k|k-1} - c)$$
$$P_{k|k} = (I - K_k H) P_{k|k-1}$$